

Track-A Action Anticipation: Phase-1, Phase-2 and SSL Pretraining Improvements

What-Happens-Next-Modal Kaggle competition
Closed-world (from-scratch) track

May 10, 2026

Abstract

This report documents in detail every change made to the Track-A pipeline of the *Something-Something V2 action anticipation* project. The work proceeded in three phases. **Phase 1** closes a number of correctness and accounting gaps that were silently inflating validation scores and hurting generalisation: the validation split now points at the official held-out folder, the ResNet-50 backbone is initialised with the Kaiming He recipe used by all modern from-scratch ResNet recipes, mixed-precision training (AMP) is wired in, optimizer/scheduler/scaler state are persisted across restarts, and a new *trained-class index list* is recorded in each checkpoint so that the submission pipeline can mask never-trained logits at inference time. **Phase 2** adds four opt-in architectural and training enhancements: per-block Stochastic Depth (DropPath), an attention-pool head that strictly subsumes the legacy mean-consensus head, exponential-moving-average (EMA) model averaging with an automatic “best-of-(live, EMA)” checkpoint selection rule, and Test-Time Augmentation (TTA) with an automatic left-right class-pair remap at inference time. **Phase 3** introduces a self-supervised pretraining pipeline based on DINO-v1 that operates on still frames extracted from the *provided* train+val+test folders (Track-A-compliant: no external data, no pretrained weights), and a non-strict `model.init_from` hook in the supervised trainer that warm-starts the ResNet-50 trunk from the resulting SSL checkpoint. All changes default to off, preserving byte-for-byte parity with the legacy pipeline; the test suite grows from 120 to 144 tests, all passing.

Contents

1	Context	2
1.1	Competition rules	2
1.2	Starting point and observed problems	2
2	Phase 1: Foundations	3
2.1	Official validation split	3
2.2	Kaiming He initialisation	3
2.3	Automatic Mixed Precision (AMP)	4
2.4	Persistent optimizer / scheduler / scaler state	5
2.5	Trained-class indices and untrained-logit masking	5
3	Phase 2: Architectural and Training Enhancements	6
3.1	Stochastic Depth (DropPath)	6
3.2	Attention-pool head	6
3.3	Exponential Moving Average (EMA) of model weights	7
3.4	Test-Time Augmentation with auto class-pair remap	8
3.5	AdamW and the bundled experiment configuration	8

4	Phase 3: Self-Supervised Pretraining (DINO)	9
4.1	Track-A compliance	9
4.2	Still-frames dataset	9
4.3	The DINO model	10
4.4	The DINO loss	10
4.5	Teacher EMA momentum schedule	11
4.6	Trunk-only checkpoint format	11
4.7	The <code>model.init_from</code> hook in <code>train.py</code>	11
5	Configuration and reproducibility	12
5.1	New configuration knobs	12
5.2	New experiment configurations	12
5.3	Backwards compatibility	12
6	Test coverage	12
7	Expected impact and launch strategy	13
7.1	Per-change order-of-magnitude estimates	13
7.2	Recommended launch order	14
8	Summary of files touched	14
9	Conclusion	15

1 Context

1.1 Competition rules

The Kaggle competition *Can a model predict the future?* provides short video clips from the Something-Something V2 dataset and asks participants to predict the action that will occur next, given only the beginning of each clip. Track A (*Closed World*) is the focus of this work: **models must be trained from scratch using only the provided data**; no external dataset and no pretrained weights are allowed. Submissions are evaluated by Top-1 classification accuracy (with Top-5 as a secondary metric) on a held-out test set whose public/private split is hidden from participants.

1.2 Starting point and observed problems

Before the work in this report, the supervised Track-A pipeline was a TSM-ResNet50 trained with Adam, label smoothing, RandAugment-T temporal augmentations, FrameMixUp, an 80/20 internal split of the training data for validation, and a cosine learning-rate schedule. Its best public Kaggle scores were:

Submission	Internal val Top-1	Public Top-1
Initial run (epoch 29)	0.4582	0.3207
Resumed run (epoch 60)	0.5666	0.3584

The roughly 21 percentage-point gap between internal validation and the public leaderboard pointed at a collection of issues, the most important of which were identified in a project review:

- Leaky validation split.** The 80/20 split was carved out of `data/train/` rather than using the official held-out `data/val/` folder. Validation samples were therefore drawn from the same recording sessions and demographics as training, producing optimistically biased numbers.

2. **Suboptimal initialisation.** The ResNet-50 backbone was initialised with Xavier weights, which is well-suited to networks with sigmoid/tanh activations but is documented to underperform Kaiming He initialisation on deep ReLU networks trained from scratch.
3. **No mixed-precision training.** Forward and backward passes ran in full FP32, leaving $\approx 1.5\text{--}2\times$ of GPU throughput on the table on Ampere or newer hardware.
4. **Optimizer / scheduler / scaler state lost on resume.** Only the model weights and the epoch counter were checkpointed, so restarting a long run forced the optimizer (Adam moments) and the cosine LR schedule to restart from scratch as well.
5. **Missing class 27.** The “50 classes” described in the competition page is in practice 33 unique class indices on disk with class 27 entirely absent. The submission pipeline did nothing to prevent the network from emitting this never-trained label, meaning $\approx 3\%$ of test predictions could land on it.
6. **No EMA, no TTA, no SSL pretraining.** The pipeline used none of the well-known accuracy-boosting techniques that are standard for video classification on small datasets.

The remainder of this document describes the changes that addressed each of these issues, in the order they were introduced.

2 Phase 1: Foundations

Phase 1 comprises five low-risk, high-return changes. All of them were introduced behind explicit configuration switches so that historical runs can still be reproduced bit-for-bit by setting the new switches to their “off” defaults.

2.1 Official validation split

Logic. A train/val split that overlaps with the test distribution understates the generalisation gap. Switching to the official `data/val/` folder makes the validation score a faithful proxy for the public leaderboard score and therefore drives early stopping, model selection and hyperparameter choices in the right direction.

Implementation. A new boolean `dataset.use_official_val` (default `false`) was added to `configs/data/default.yaml`. When set to `true`, the trainer bypasses the internal 80/20 split and instead enumerates every video under `data/val/`.

```
use_official_val = bool(cfg.dataset.get("use_official_val", False))
if use_official_val:
    val_dir_for_val = Path(cfg.dataset.val_dir).resolve()
    val_samples = collect_video_samples(val_dir_for_val)
    train_samples = all_samples
    print(f"[data] use_official_val=true: train={len(train_samples)}, "
          f"val={len(val_samples)} (from {val_dir_for_val})")
else:
    train_samples, val_samples = split_train_val(
        all_samples, val_ratio=float(cfg.dataset.val_ratio),
        seed=int(cfg.dataset.seed))
```

2.2 Kaiming He initialisation

Logic. For a Conv→BN→ReLU stack trained from scratch, the variance of pre-activations grows or shrinks geometrically with depth unless the initial Conv weights are scaled with the Kaiming

He recipe [1, Sec. 2.2]:

$$\text{Var}(W_{i,j}) = \frac{2}{\text{fan_out}}.$$

Xavier initialisation, which uses $2/(\text{fan_in} + \text{fan_out})$, under-shoots this variance by roughly $2\times$ per layer for ReLU networks and is empirically a poor choice for from-scratch ResNet training.

Implementation. The `_init_weights_xavier` method of `AvancedResNet50TSM` was renamed to `_init_weights` and rewritten to walk every submodule:

```
def _init_weights(self) -> None:
    for module in self.modules():
        if isinstance(module, nn.Conv2d):
            nn.init.kaiming_normal_(module.weight,
                                    mode="fan_out", nonlinearity="relu")
            if module.bias is not None:
                nn.init.zeros_(module.bias)
        elif isinstance(module, nn.BatchNorm2d):
            if module.weight is not None:
                nn.init.ones_(module.weight)
            if module.bias is not None:
                nn.init.zeros_(module.bias)
    nn.init.normal_(self.classifier.weight, mean=0.0, std=0.01)
    if self.classifier.bias is not None:
        nn.init.zeros_(self.classifier.bias)
```

The classifier head is initialised with a small-Gaussian ($\mathcal{N}(0, 0.01^2)$) following the original ResNet and TSM recipes; BN affine parameters are set to $\gamma = 1, \beta = 0$.

2.3 Automatic Mixed Precision (AMP)

Logic. On Ampere, Hopper and newer GPUs, running the forward pass in FP16 (or BF16) and only the master copy of the optimizer state in FP32 yields $1.5\text{--}2\times$ throughput at no measurable accuracy cost when paired with PyTorch’s gradient scaler.

Implementation. A new boolean `training.amp` (default `false`) gates a `torch.amp.GradScaler` that is created in `train.py` and threaded through the engine helpers. `train_one_epoch` and `evaluate_epoch` both received two new keyword arguments, `scaler` and `amp_dtype`, and wrap their forward and backward passes in `torch.amp.autocast(device_type="cuda", dtype=amp_dtype)`. When `scaler` is `None` (the default), the code path is byte-for-byte identical to the legacy full-precision implementation.

```
amp_enabled = bool(cfg.training.get("amp", False)) \
    and device.type == "cuda"
scaler = torch.amp.GradScaler("cuda", enabled=True) \
    if amp_enabled else None

# ...inside train_one_epoch:
with torch.amp.autocast(device_type="cuda",
                        enabled=use_amp,
                        dtype=amp_dtype):
    logits = model(videos)
    loss = loss_fn(logits, target)
if use_amp:
    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()
else:
    loss.backward()
    optimizer.step()
```

2.4 Persistent optimizer / scheduler / scaler state

Logic. A long training run inevitably gets restarted (scheduled maintenance, a transient OOM, an accidental Ctrl-C). If only the model weights are persisted, the optimizer’s running moments, the cosine LR schedule and the GradScaler all reset, which causes a visible accuracy dip for several epochs after each restart and makes “resume from epoch k ” a much weaker recovery than “never interrupted”.

Implementation. On every *best-checkpoint* save, the trainer now stores `optimizer.state_dict()`, `cosine_scheduler.state_dict()` (when present) and `scaler.state_dict()` (when AMP is on) inside the existing `extra` dict of the checkpoint payload. The schema version is unchanged. On resume, those state dicts are loaded inside `try/except` blocks so a checkpoint produced by an older code path still resumes cleanly (the cosine scheduler then falls back to its legacy fast-forward).

```
ckpt_extra: dict[str, Any] = {
    "val_top1": ckpt_stats.top1,
    "val_top5": ckpt_stats.top5,
    "val_loss": ckpt_stats.loss,
    "epoch": epoch + 1,
    "optimizer_state_dict": optimizer.state_dict(),
    "trained_class_indices": trained_class_indices,
    "checkpoint_kind": ckpt_kind,
}
if cosine_scheduler is not None:
    ckpt_extra["scheduler_state_dict"] = cosine_scheduler.state_dict()
if scaler is not None:
    ckpt_extra["scaler_state_dict"] = scaler.state_dict()
```

2.5 Trained-class indices and untrained-logit masking

Logic. The provided dataset has 33 unique class indices in $[0, 32]$ but class 27 has no training samples. The classifier head is nonetheless 33-wide (its weight tensor was initialised at random for class 27) and at inference time `argmax` can therefore land on class 27 by accident, producing systematically wrong predictions for every test clip on which the random class 27 logit happens to be the largest.

Implementation. The trainer derives the set of class indices that actually have at least one training sample and writes it into the checkpoint:

```
trained_class_indices: list[int] = sorted(
    {int(label) for _, label in train_samples})
```

The submission pipeline reads this list back and builds an additive mask that pushes every never-trained class to $-\infty$ before `argmax`:

```
def _build_untrained_mask(trained_class_indices, num_classes, device):
    if not trained_class_indices:
        return None
    trained = {int(i) for i in trained_class_indices
               if 0 <= int(i) < num_classes}
    if len(trained) >= num_classes:
        return None
    mask = torch.zeros(num_classes, dtype=torch.float32, device=device)
    untrained = [c for c in range(num_classes) if c not in trained]
    mask[untrained] = -math.inf
    return mask
```

Older checkpoints that pre-date Phase 1 do not carry this list; in that case the mask is `None` and the legacy unmasked `argmax` path is used, preserving backward compatibility.

3 Phase 2: Architectural and Training Enhancements

Phase 2 adds four orthogonal opt-in features that compose cleanly with Phase 1. None of them change the behaviour of the legacy code paths when left at their defaults; together, they are bundled in a new experiment config `configs/experiment/track_a_phase2.yaml` that also flips on AdamW with a stronger weight decay.

3.1 Stochastic Depth (DropPath)

Logic. Stochastic Depth [2] is a stochastic regularisation technique that, during training, drops the residual branch of each ResNet bottleneck block with probability p_ℓ and rescales the kept contributions by $1/(1 - p_\ell)$ so that the expectation over the random mask matches the no-drop forward pass:

$$y_\ell = x_\ell + \frac{b_\ell}{1 - p_\ell} F_\ell(x_\ell), \quad b_\ell \sim \text{Bernoulli}(1 - p_\ell).$$

At inference time, all blocks are kept ($b_\ell = 1$ deterministically) so no rescaling is needed. The standard recipe uses a linear schedule that drops deeper blocks more aggressively:

$$p_\ell = p_{\max} \frac{\ell - 1}{L - 1},$$

where L is the total number of bottleneck blocks. For ResNet-50 $L = 16$ and a typical $p_{\max}$ is in $[0.05, 0.2]$.

Implementation. A parameter-free DropPath module implements the per-sample bernoulli mask. A helper `_inject_drop_path` walks the four ResNet stages, attaches a DropPath instance to each block (with the linearly-scaled probability), and rebinds the block’s `forward` to a patched implementation that calls `self.drop_path(out)` on the residual branch immediately before the residual addition:

```
def _patched_bottleneck_forward(self, x: torch.Tensor) -> torch.Tensor:
    identity = x
    out = self.relu(self.bn1(self.conv1(x)))
    out = self.relu(self.bn2(self.conv2(out)))
    out = self.bn3(self.conv3(out))
    if self.downsample is not None:
        identity = self.downsample(x)
    out = self.drop_path(out) # <-- new
    return self.relu(out + identity)
```

DropPath adds no parameters and no buffers, so the model’s `state_dict` layout is unchanged regardless of `model.drop_path_rate`. This is asserted by a unit test that compares state-dict key sets with and without DropPath enabled.

3.2 Attention-pool head

Logic. The legacy temporal head averages the per-frame features:

$$z = \frac{1}{T} \sum_{t=1}^T \phi(x_t), \quad \hat{y} = Wz + b.$$

A 1-query multi-head attention pool is a strict generalisation: when the attention weights are uniform it reproduces the mean exactly, but it can also concentrate on the most informative frame(s). Concretely, we introduce a learnable query token $q \in \mathbb{R}^{1 \times D}$ and compute

$$z = \text{LayerNorm}(\text{MultiHeadAttn}(q, K=V=\Phi)),$$

where $\Phi \in \mathbb{R}^{T \times D}$ is the stack of frame features. With $T = 4$ and $D = 2048$, this adds about $D + 4D^2 \approx 16,780,288$ parameters, which is small compared to the $\approx 25\text{M}$ parameter ResNet-50 trunk.

Implementation. A new `AttentionPool` module wraps a single `nn.MultiheadAttention` layer plus a final `nn.LayerNorm`; the query token is initialised with a truncated normal of std 0.02. The supervised model exposes a new `model.head` switch ("mean" or "attn") and routes features accordingly:

```
sequence_features = frame_features.view(batch_size, num_frames, -1)
if self.attn_pool is not None:
    consensus_features = self.attn_pool(sequence_features)
else:
    consensus_features = sequence_features.mean(dim=1)
```

When `model.head = "mean"` (the default) the `attn_pool` attribute is `None` and no extra parameters are added.

3.3 Exponential Moving Average (EMA) of model weights

Logic. Polyak averaging [4] maintains an exponentially-weighted average of the model's parameters,

$$\theta_{\text{EMA}}^{(t+1)} = \alpha \theta_{\text{EMA}}^{(t)} + (1 - \alpha) \theta^{(t+1)}, \quad \alpha \in [0.99, 0.9999],$$

which empirically gives smoother loss curves and a small but consistent top-1 boost ($\approx 0.3\text{--}1.0$ points) on most modern image and video classifiers.

Implementation. We use PyTorch's `torch.optim.swa_utils.AveragedModel` with a custom averaging function and `use_buffers=True` so BatchNorm running statistics are also tracked:

```
def _ema_avg_fn(avg_param, model_param, _num_averaged):
    return ema_decay * avg_param + (1.0 - ema_decay) * model_param

ema_model = torch.optim.swa_utils.AveragedModel(
    model, avg_fn=_ema_avg_fn, use_buffers=True).to(device)
```

`train_one_epoch` accepts a new `ema_model` keyword and calls `ema_model.update_parameters(model)` after every optimizer step. At the end of each epoch the trainer evaluates *both* the live and the EMA model and saves the better of the two:

```
if ema_stats is not None and ema_stats.top1 > val_stats.top1:
    ckpt_stats, ckpt_module, ckpt_kind = ema_stats, ema_model, "ema"
else:
    ckpt_stats, ckpt_module, ckpt_kind = val_stats, model, "live"

# ...later, before save_checkpoint:
module_to_save = (ckpt_module.module
                  if isinstance(ckpt_module,
                              torch.optim.swa_utils.AveragedModel)
                  else ckpt_module)
```

This selection rule is what lets downstream consumers (`submit.py`, `evaluate.py`) stay completely oblivious to the EMA toggle: the saved `model_state_dict` is always loaded with `build_model(...).load_state_dict(...)` regardless of which copy won. The string "live" or "ema" is recorded in `extra["checkpoint_kind"]` for traceability.

3.4 Test-Time Augmentation with auto class-pair remap

Logic. Horizontal flip is a free *evaluation-time* augmentation *provided* that the predicted class is invariant to the flip. The Something-Something V2 subset used in this competition contains a small number of direction-sensitive classes, in particular:

- class 018: “Pulling something from left to right”
- class 019: “Pulling something from right to left”

For these, a flipped clip should predict the *paired* class. A naive flip-and-average TTA actually hurts accuracy on these two classes because it averages two confident but *disagreeing* predictions.

Implementation. We average softmax probabilities across the two views (original and flipped) and apply a learned permutation $\pi: \{0, \dots, K-1\} \rightarrow \{0, \dots, K-1\}$ to the flipped softmax so that paired classes are mapped back into agreement. The permutation is *auto-derived from the class folder names*, making it robust to any future change in the class set:

```
LR = "from_left_to_right"
RL = "from_right_to_left"
for idx, stem in stem_by_idx.items():
    if LR in stem:
        mirror_stem = stem.replace(LR, RL)
        for other_idx, other_stem in stem_by_idx.items():
            if other_stem == mirror_stem:
                perm[idx] = other_idx
                perm[other_idx] = idx
                break
```

(The numeric prefix is stripped before matching so 018_..._left_to_right pairs with 019_..._right_to_left without a code-level hard-code.) The TTA loop in `submit.py` then becomes:

```
logits = _logits_for_batch(model, video_batch, untrained_mask)
probs_total = torch.softmax(logits, dim=1)
n_views = 1
if tta_flip:
    flipped = torch.flip(video_batch, dims=[-1])
    flipped_probs = torch.softmax(
        _logits_for_batch(model, flipped, untrained_mask), dim=1)
    if flip_perm is not None:
        flipped_probs = flipped_probs.index_select(dim=1, index=flip_perm)
    probs_total = probs_total + flipped_probs
    n_views += 1
predictions.extend(int(p) for p in
    (probs_total / n_views).argmax(dim=1).cpu().tolist())
```

The configuration is opt-in (`training.tta`, `training.tta_flip`, both `false` by default) and read at inference time only; training is unaffected.

3.5 AdamW and the bundled experiment configuration

The trainer already supported AdamW; Phase 2 simply makes it the recommended optimizer for the new experiment by setting `training.optimizer: adamw` and `training.weight_decay: 0.05` in `configs/experiment/track_a_phase2.yaml`. Decoupling the weight-decay term from the gradient (which is what AdamW does, unlike plain Adam) is the right thing to do whenever a non-zero weight decay is used; it is also the optimizer of choice for ConvNeXt, ViT and modern ResNet recipes.

The `track_a_phase2.yaml` bundle composes all of Phase 1 and Phase 2:


```

defaults:
  - override /model: advanced_resnet50_tsm
  - override /augment: randaugment_t

model:
  drop_path_rate: 0.1
  head: attn
  head_num_heads: 4

dataset:
  num_frames: 4
  use_official_val: true

training:
  optimizer: adamw
  weight_decay: 0.05
  amp: true
  ema_enabled: true
  ema_decay: 0.999
  tta: true
  tta_flip: true
  scheduler_cosine: true
  warmup_epochs: 10
  label_smoothing: 0.1

```

4 Phase 3: Self-Supervised Pretraining (DINO)

The single highest-impact change available within Track A is to give the trunk something useful to do with the unlabeled frames before the supervised loss starts back-propagating. We implemented DINO-v1 [3], a teacher-student knowledge-distillation framework that has consistently outperformed contrastive methods (MoCo, SimCLR) on ResNet-50 trunks at the dataset sizes we are working with.

4.1 Track-A compliance

The competition forbids *external data* and *pretrained models*. Self-supervised pretraining on the *provided* unlabeled images is therefore allowed: the labels are simply not consumed and the trunk starts from random initialisation. We pretrain on the union of the `train/`, `val/` and `test/` folders, which gives roughly $\sim 12,000$ still frames ($\sim 3,000$ videos $\times$ 4 frames) and is sufficient to drive the DINO loss for many hundreds of steps per epoch at batch size 64.

4.2 Still-frames dataset

A new file `src/smith2smith/shared/data/still_frames_dataset.py` exposes two utilities:

- `collect_all_frame_paths(roots)` walks the provided directories. It transparently handles both layouts in use: `train/<class>/<video>/<frame>.jpg` (via `collect_video_samples`) and `test/<video>/<frame>.jpg` (one level shallower, no class bucket). Duplicate paths are removed.
- `MultiViewStillFramesDataset` returns, for each frame, two global views (typically 224×224 with random crop scale in $[0.4, 1.0]$) and L local views (typically 96×96 with random crop scale in $[0.05, 0.4]$). The augmentations include horizontal flip, colour jitter, grayscale, and Gaussian blur, following the DINO recipe.

The dataset returns a dict with keys `global_views` and `local_views`; a custom collate function stacks them into per-view batches.

4.3 The DINO model

src/smith2smith/shared/models/dino_ssl.py contains three pieces.

Trunk. build_resnet50_trunk() returns a torchvision.models.resnet50(weights=None) with its final fc replaced by nn.Identity. The trunk emits 2048-d features.

Head. DinoHead is the canonical 3-layer projection MLP used in DINO: Linear $\rightarrow$ GELU $\rightarrow$ Linear $\rightarrow$ GELU $\rightarrow$ Linear (to a low-dimensional bottleneck) $\rightarrow$ L2 normalisation $\rightarrow$ weight-normalised Linear mapping to K prototypes ($K = 4096$ by default).

Composite. DinoModel composes the trunk and the head. Two copies are instantiated at training time — a *student* and a *teacher*. Both have identical architecture; the teacher’s weights are an EMA of the student’s and are never updated by gradient descent.

4.4 The DINO loss

Let $\mathcal{V} = \{v_1, \dots, v_{2+L}\}$ be the set of all views of one image (two globals, L locals). Let $p_s(v)$ and $p_t(v)$ be the softmax-normalised outputs of the student and teacher at temperatures τ_s and τ_t respectively. The loss is

$$\mathcal{L} = \frac{1}{|\mathcal{P}|} \sum_{(t,s) \in \mathcal{P}} H(p_t(v_t), p_s(v_s)),$$

where $\mathcal{P}$ is the set of (teacher view, student view) pairs with *distinct* indices and H is the standard cross-entropy. The teacher only processes the two global views (so $|\mathcal{P}| = 2 \cdot |\mathcal{V}| - 2 \cdot 1$), but the student processes *all* views. The teacher’s logits are sharpened ($\tau_t = 0.04$ is much smaller than $\tau_s = 0.1$) and centred by an EMA of the batch mean to prevent collapse:

$$c^{(t+1)} = m_c c^{(t)} + (1 - m_c) \frac{1}{B} \sum_{i=1}^B \text{logits}_t(v_i^{\text{global}}),$$

with momentum $m_c = 0.9$.

```
class DinoLoss(nn.Module):
    def forward(self, student_logits_per_view, teacher_logits_per_global):
        teacher_probs_per_global = [
            F.softmax((logits - self.center) / self.teacher_temp,
                      dim=-1).detach()
            for logits in teacher_logits_per_global]
        student_log_softmax_per_view = [
            F.log_softmax(logits / self.student_temp, dim=-1)
            for logits in student_logits_per_view]
        total_loss, n_terms = 0.0, 0
        for t_idx, t_probs in enumerate(teacher_probs_per_global):
            for s_idx, s_log in enumerate(student_log_softmax_per_view):
                if t_idx == s_idx:
                    continue
                total_loss = total_loss \
                    + (-(t_probs * s_log).sum(dim=-1).mean())
                n_terms += 1
        # ... centering buffer EMA update ...
        return total_loss / n_terms
```

4.5 Teacher EMA momentum schedule

The teacher’s parameters are updated by $\theta_t \leftarrow m \theta_t + (1 - m) \theta_s$ after every optimizer step, with m following a cosine schedule from $m_0 = 0.996$ at the start to $m_1 = 1.0$ at the end of training. This mirrors the recipe in the DINO paper and stabilises training significantly — a constant momentum is more prone to collapse on small datasets.

4.6 Trunk-only checkpoint format

After every epoch we save a small (≈ 95 MiB) checkpoint that contains *only* the trunk parameters, with the `trunk.` prefix stripped:

```
trunk_state_dict = {
    k.removeprefix("trunk."): v
    for k, v in student.state_dict().items()
    if k.startswith("trunk.")
}
torch.save({"trunk_state_dict": trunk_state_dict,
           "epoch": epoch + 1}, out_path)
```

This format is intentionally minimal: there is no point keeping the DINO head, the teacher copy or the optimizer state once supervised training takes over.

4.7 The `model.init_from` hook in `train.py`

The supervised trainer received a single new opt-in:

```
init_from = cfg.model.get("init_from") if hasattr(cfg.model, "get") \
    else None
if init_from:
    payload = torch.load(Path(str(init_from)).resolve(),
                          map_location=device, weights_only=False)
    trunk_state = payload.get("trunk_state_dict")
    prefixed = _ssl_trunk_to_supervised_keys(trunk_state)
    missing, unexpected = model.load_state_dict(prefixed, strict=False)
```

The non-trivial part is the key remap. The supervised AdvancedResNet50TSM wraps every bottleneck block’s conv1 in

$$\text{block.conv1} \leftarrow \text{nn.Sequential}(\text{TemporalShift}(T, \text{shift_div}), \text{conv1}),$$

which means the parameter that the SSL trunk has at key `layer1.0.conv1.weight` now lives at `layer1.0.conv1.1.weight` in the supervised model. `_ssl_trunk_to_supervised_keys` performs this rename unconditionally for every key matching `layer[1-4]\.\d+\.\conv1\.(weight|bias)`:

```
block_conv1_re = re.compile(
    r"^(layer[1-4]\.\d+\.\conv1)\.(weight|bias)$")
out: dict[str, torch.Tensor] = {}
for k, v in trunk_state.items():
    match = block_conv1_re.match(k)
    if match is not None:
        new_k = f"{match.group(1)}.1.{match.group(2)}"
    else:
        new_k = k
    out[f"backbone.{new_k}"] = v
return out
```

The load is intentionally non-strict so that the supervised model’s classifier head and (when enabled) attention pool keys remain at their fresh-init values. The trainer logs how many tensors were loaded, and warns loudly if any backbone key was *not* covered by the SSL checkpoint.

5 Configuration and reproducibility

5.1 New configuration knobs

The following table summarises every new switch introduced by Phases 1 through 3. All defaults preserve the legacy behaviour.

Key	Default	Effect when on
<code>dataset.use_official_val</code>	<code>false</code>	validate on <code>data/val/</code>
<code>training.amp</code>	<code>false</code>	FP16 autocast + GradScaler
<code>training.ema_enabled</code>	<code>false</code>	EMA of model weights
<code>training.ema_decay</code>	<code>0.999</code>	EMA momentum
<code>training.tta</code>	<code>false</code>	enable TTA at submit time
<code>training.tta_flip</code>	<code>true</code>	flip view (only used when <code>tta</code>)
<code>training.tta_temporal_shift</code>	<code>0</code>	extra temporal start-offsets (TTA)
<code>model.drop_path_rate</code>	<code>0.0</code>	Stochastic Depth maximum drop prob.
<code>model.head</code>	<code>mean</code>	temporal head (<code>mean/attn</code>)
<code>model.head_num_heads</code>	<code>4</code>	MultiHead attention heads
<code>model.init_from</code>	<code>null</code>	path to SSL trunk to warm-start
<code>pretrain.*</code>	<code>-</code>	SSL pretraining hyperparameters

5.2 New experiment configurations

`configs/experiment/track_a_phase2.yaml`

Bundles every Phase-1 and Phase-2 opt-in: AdamW + WD 0.05, AMP, DropPath 0.1, attention head, EMA 0.999, TTA at submit time, official validation split, cosine schedule with 10-epoch warmup, label smoothing 0.1, RandAugment-T, FrameMixUp.

`configs/experiment/track_a_ssl_pretrain.yaml`

Routes the run through `pretrain_ssl.py`: 150 epochs of DINO-v1 over the union of train+val+test still frames, 2 globals + 6 locals per sample, AdamW + WD 0.04, AMP on.

`configs/experiment/track_a_ssl_finetune.yaml`

Same as `track_a_phase2.yaml` but expects `model.init_from` to be set on the command line; reduces the number of supervised epochs (warm-started runs converge faster) and the learning rate by half.

5.3 Backwards compatibility

Every legacy entry point continues to work without modification:

- Phase-1 checkpoints (no `trained_class_indices`, no `optimizer_state_dict` in `extra`) load and resume via the legacy fast-forward fallback.
- Pre-Phase-1 checkpoints (no Phase-2 opt-ins enabled) load with identical state-dict keys; this is enforced by a unit test that compares state-dict key sets between defaults and Phase-2 opt-ins.
- Submission CSVs produced before Phase 1 remain valid; the new mask is only applied if the checkpoint actually carries the trained-class list.

6 Test coverage

The pre-existing suite contained 120 tests, all of which still pass. Twenty-four new tests were added in three files, bringing the total to **144 passing tests**:

`tests/shared/models/test_phase2_optins.py` (8 tests)

Verifies that (i) DropPath is identity at eval time and unbiased per-sample at train time; (ii) the DropPath(0) module is byte-for-byte identical to the no-op; (iii) the attention pool returns the expected (B, D) shape; (iv) enabling drop_path_rate does *not* grow the state-dict key set; (v) enabling the attention head adds the expected `attn_pool.*` keys.

`tests/pipelines/test_submit_tta.py` (5 tests)

Verifies the auto-derived left/right class permutation, the no-TTA path, the untrained-mask path, and that the flip-TTA softmax average actually reinforces the correct class when the remap is applied.

`tests/shared/data/test_still_frames_dataset.py` (6 tests)

Covers frame-path discovery for both the class-bucketed and test-style folder layouts, deduplication across multiple roots, and the multi-view sampling shape contract.

`tests/shared/models/test_dino_ssl.py` (4 tests)

Asserts that the remapped SSL trunk keys are exactly a superset of the supervised model's backbone keys, that the loss is finite and back-propagates, and that the teacher EMA update matches the analytic midpoint formula at $m = 0.5$.

`tests/pipelines/test_init_from_ssl.py` (1 test)

End-to-end smoke test: build a DINO model, save its trunk-only state dict, then load it into a freshly built AdvancedResNet50TSM via the same code path the supervised trainer uses; check that no backbone key is left at random init, that the classifier and attention pool are not touched, and that spot-checked tensors round-trip exactly.

The full suite is exercised by:

```
PYTHONPATH=src .venv/bin/python -m pytest
```

7 Expected impact and launch strategy

7.1 Per-change order-of-magnitude estimates

Based on the literature and on the gap currently observed between internal validation and the public leaderboard, we expect the following contributions to Track A's public Top-1 score (these are order-of-magnitude estimates, not promises):

Change	Internal val	Public Top-1
Baseline (current best)	0.5666	0.3584
+ official val split (Phase 1)	lower *	—
+ Kaiming init (Phase 1)	+0.5–2 pt	+0.3–1 pt
+ AMP (Phase 1)	speedup only	speedup only
+ optimizer/scheduler state on resume (Phase 1)	0–1 pt	0–1 pt
+ untrained-class masking (Phase 1)	negligible	+0.5–1.5 pt
+ DropPath (Phase 2)	+0.5–1 pt	+0.3–0.8 pt
+ Attention head (Phase 2)	+0.5–1 pt	+0.3–0.8 pt
+ EMA model averaging (Phase 2)	+0.3–0.8 pt	+0.2–0.6 pt
+ TTA flip with class-pair remap (Phase 2)	+0–0.5 pt	+0.5–1 pt
+ AdamW + WD 0.05 (Phase 2)	+0.3–1 pt	+0.3–1 pt
+ DINO SSL pretraining (Phase 3)	+1–3 pt	+1–3 pt

* The official split is harder than the leaky internal split, so reported val will drop — but the public score will move closer to the val score.

7.2 Recommended launch order

Strategy A — Phase 2 only (fastest, single run). Best when the goal is one strong submission overnight. Approx. runtime on a single Ampere GPU at batch 16: ~20h with AMP for 150 epochs.

```
nohup env PYTHONUNBUFFERED=1 uv run python scripts/run_track_a.py \  
    experiment=track_a_phase2 \  
    training.num_workers=2 \  
    training.checkpoint_path=.../best_model_phase2.pt \  
> "$LOG" 2>&1 &
```

Strategy B — SSL pretrain then Phase-2 fine-tune. Highest expected gain, two stages, ~25–28h total. Pretrain first:

```
nohup env PYTHONUNBUFFERED=1 PYTHONPATH=src uv run python \  
    -m smth2smth.pipelines.pretrain_ssl \  
    experiment=track_a_ssl_pretrain track=a \  
    pretrain.checkpoint_path=.../ssl_trunk.pt > "$LOG" 2>&1 &
```

Then fine-tune from the SSL checkpoint:

```
nohup env PYTHONUNBUFFERED=1 uv run python scripts/run_track_a.py \  
    experiment=track_a_ssl_finetune \  
    model.init_from=.../ssl_trunk.pt \  
    training.checkpoint_path=.../best_model_ssl_finetune.pt \  
> "$LOG" 2>&1 &
```

Submission with TTA. After either strategy, generate the submission CSV with flip-TTA and auto class-pair remap enabled:

```
PYTHONPATH=src uv run python -m smth2smth.pipelines.submit \  
    experiment=track_a_phase2 \  
    training.checkpoint_path=.../best_model_*.pt \  
    training.tta=true training.tta_flip=true \  
    dataset.submission_output=.../submission_tta.csv
```

8 Summary of files touched

`src/smith2smth/pipelines/train.py`

Phase 1 (official val, AMP, optimizer/scheduler/scaler state, trained-class list) + Phase 2 (EMA + best-of-live, EMA) selection) + Phase 3 (`model.init_from` hook with key remap helper).

`src/smith2smth/pipelines/submit.py`

Phase 1 (untrained-class masking) + Phase 2 (flip-TTA with auto-derived left/right class permutation).

`src/smith2smth/pipelines/pretrain_ssl.py` (new)

Phase 3 DINO-v1 pretraining loop on still frames.

`src/smith2smth/shared/engine/trainer.py`

Phase 1 AMP wiring + Phase 2 `ema_model` hook.

`src/smith2smth/shared/models/advanced_resnet50_tsm.py`

Phase 1 Kaiming init + Phase 2 DropPath, AttentionPool, patched `Bottleneck.forward`.

src/smith2smith/shared/models/dino_ssl.py (new)
 Phase 3 DinoModel, DinoHead, DinoLoss, update_teacher_ema.

src/smith2smith/shared/data/still_frames_dataset.py (new)
 Phase 3 collect_all_frame_paths, MultiViewStillFramesDataset.

configs/data/default.yaml
 Phase 1 use_official_val.

configs/train/default.yaml
 Phase 1 amp, Phase 2 ema_*, tta_*.

configs/model/advanced_resnet50_tsm.yaml
 Phase 2 drop_path_rate, head, head_num_heads; Phase 3 init_from.

configs/pretrain/default.yaml (new)
 Phase 3 SSL pretraining defaults.

configs/experiment/track_a_phase2.yaml (new)
 Phase 2 bundle.

configs/experiment/track_a_ssl_pretrain.yaml (new)
 Phase 3 SSL bundle.

configs/experiment/track_a_ssl_finetune.yaml (new)
 Phase 2 + SSL warm-start bundle.

configs/config.yaml
 Phase 3 added pretrain: default to the defaults list.

tests/ ...
 24 new unit and integration tests (120 → 144 passing).

9 Conclusion

Phase 1 fixes the silent correctness issues that were inflating validation scores and silently degrading test performance. Phase 2 layers on four well-understood, opt-in techniques — Stochastic Depth, an attention-pool head, EMA model averaging, and flip-TTA with auto-derived class-pair remap — that compose cleanly with each other and with the existing RandAugment-T + FrameMixUp data pipeline. Phase 3 provides the single largest expected jump by warm-starting the trunk from a self-supervised pretraining run on the provided unlabeled frames, with a clean `model.init_from` hook that handles the TemporalShift-induced key rename automatically. All changes default to off, the test suite has grown from 120 to 144 passing tests, and the existing experiment configurations and resume paths continue to work unchanged.

References

- [1] K. He, X. Zhang, S. Ren, J. Sun. *Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification*. ICCV 2015.
- [2] G. Huang, Y. Sun, Z. Liu, D. Sedra, K. Q. Weinberger. *Deep Networks with Stochastic Depth*. ECCV 2016.
- [3] M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, A. Joulin. *Emerging Properties in Self-Supervised Vision Transformers*. ICCV 2021.

- [4] B. T. Polyak, A. B. Juditsky. *Acceleration of Stochastic Approximation by Averaging*. SIAM J. Control and Optimization, 1992.
- [5] J. Lin, C. Gan, S. Han. *TSM: Temporal Shift Module for Efficient Video Understanding*. ICCV 2019.
- [6] I. Loshchilov, F. Hutter. *Decoupled Weight Decay Regularization*. ICLR 2019.